

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

Отчет по блоку задач №3 (Яндекс.Контест)

Выполнил:

Студент группы Р3209

Евграфов А. А.

Преподаватель:

Косяков М. С.

Санкт-Петербург, 2026 г.

Задачи из 3-его блока Яндекс.Контеста

Задача №9 «I. Машины»

Описание

N машинок лежат на полке. На полу может быть не более K машинок. Петя играет последовательно: в каждый момент ему нужна машинка a_i . Если нужная машинка уже на полу — он берёт её сам. Если её нет на полу — мама достаёт её с полки и одновременно ставит на полку любую машинку с пола. Если мест на полу меньше K , мама просто ставит новую машинку. Начальное состояние: все машинки на полке (пол пуст). Мама знает всю будущую последовательность наперёд. Нужно минимизировать количество операций мамы.

Ограничение времени: 1 секунда.

Ограничение памяти: 256 Мб.

Решение

Проходим по запросам и при необходимости вытеснить машинку с пола выбираем ту, которая потребуется Пете позже всего в будущем (или не потребуется никогда).

Доказательство

Возьмём гипотетический оптимальный алгоритм OPT и покажем, что его можно шаг за шагом преобразовать в наш жадный алгоритм $OURs$, ни на одном шаге не увеличивая количество промахов.

Пусть $S = (a_1, a_2, \dots, a_P)$ — последовательность запросов к машинкам. S_{cache} — это множество машинок, которые лежат на полу прямо сейчас. Предположим, что OPT и $OURs$ работают одинаково на первых $i - 1$ запросах, но на i -м запросе их действия расходятся первый раз.

На шаге i происходит запрос машинки $r = a_i$. Расхождение алгоритмов возможно только если $r \notin S_{cache}$ и нет свободных мест на полу. В этот момент $OURs$ вытесняет машинку f (чья следующая встреча дальше всего в будущем), а OPT вытесняет машинку g ($g \neq f$). Создадим алгоритм OPT' , который на шаге i поступает так же, как $OURs$ (вытесняет f). Сразу после шага i состояния пола (кэша) будут следующими:

$$C_{OPT} = (S_{cache} \setminus \{g\}) \cup \{r\}$$

$$C_{OPT'} = (S_{cache} \setminus \{f\}) \cup \{r\}$$

Пусть $C = S_{cache} \setminus \{f, g\} \cup \{r\}$ — их общая часть. Тогда $C_{OPT} = C \cup \{f\}$, а $C_{OPT'} = C \cup \{g\}$.

Инвариант: наборы машинок на полу у алгоритмов отличаются ровно на один элемент. Пусть уникальный элемент алгоритма OPT как x , а уникальный элемент алгоритма OPT' как y . Сразу после i -го шага $x = f$, $y = g$.

Рассмотрим очередной запрос req в момент времени $t > i$. Возможны 4 случая:

- Случай 1: $req \in C_{current}$. Оба алгоритма делают попадание, инвариант сохраняется.
- Случай 2: $req \notin C_{current}$ и $req \neq x$, $req \neq y$. Оба получают промах.

- Если OPT вытесняет машинку $e \in C_{current}$, то OPT' вытесняет её же (инвариант сохранен).
 - Если OPT вытесняет свой уникальный элемент x , то OPT' вытесняет свой уникальный элемент y . С этого момента множества машинок на полу становятся идентичными и OPT' начинает полностью копировать OPT .
- Случай 3: $req = y$. У OPT промах, у OPT' попадание.
 - Если OPT вытеснит x , кэши сливаются (при этом OPT' строго обогнал OPT на 1 промах).
 - Если OPT вытеснит элемент $e \in C_{current}$, инвариант сохранится, но с новой парой различий (x, e) . То есть теперь у OPT' уникальным элементом становится e .
 - Случай 4: $req = x$. Разобьём этот случай на две фазы в зависимости от того, когда он наступает:
 1. До наступления Случая 3: по определению алгоритма $OURs$, на шаге i элемент f (изначальный x) был выбран потому, что его следующее использование находится дальше, чем использование любого другого элемента (включая y). Следовательно, запрос $req = y$ гарантированно произойдёт раньше, чем запрос $req = x$. Эта ситуация невозможна.
 2. После наступления Случая 3: из-за случая 3 пара уникальных элементов сдвинулась. Теперь OPT имеет x , а OPT' имеет какой-то другой элемент e . Теперь запрос $req = x$ возможен. В этой ситуации у OPT — попадание, а у OPT' — промах. Однако в случае 3 алгоритм OPT' сэкономил одну операцию. OPT' вытесняет свой уникальный элемент e и загружает запрошенный x . В результате суммарное количество промахов у алгоритмов на данном отрезке выравнивается, а их кэши становятся равны $C_{current} \cup \{x\}$.

Алгоритмическая сложность

- По времени: $O(P \log(K))$
- По памяти: $O(P)$

Код

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(nullptr);
8
9     int n, k, p;
10    cin >> n >> k >> p;
11
12    vector<int> a(p);
13    vector<int> b(n + 1);
14    vector<int> m(p);

```

```

15
16 for (int i = 0; i < p; ++i) {
17     cin >> m[i];
18 }
19
20 for (int i = 0; i <= n; ++i) {
21     b[i] = 5800001;
22 }
23
24 for (int i = p - 1; i >= 0; --i) {
25     int c = m[i];
26     a[i] = b[c];
27     b[c] = i;
28 }
29
30 set<pair<int, int>> s;
31 vector<bool> flags(n + 1);
32
33 for (int i = 0; i <= n; ++i) {
34     flags[i] = true;
35 }
36
37 int count = 0;
38 for (int i = 0; i < p; ++i) {
39     if ((int)s.size() < k && flags[m[i]]) {
40         s.insert(make_pair(-a[i], m[i]));
41         flags[m[i]] = false;
42         count++;
43     } else if (!flags[m[i]]) {
44         s.erase({-i, m[i]});
45         s.insert({-a[i], m[i]});
46     } else if ((int)s.size() == k && flags[m[i]]) {
47         count++;
48         flags[s.begin()->second] = true;
49         flags[m[i]] = false;
50         s.erase(s.begin());
51         s.insert(make_pair(-a[i], m[i]));
52     }
53 }
54
55 cout << count;
56 return 0;
57 }

```

Задача №10 «J. Гоблины и очереди»

Описание

Гоблины посещают шаманов и выстраиваются в очередь. Обычные гоблины встают в конец очереди (+ i). Привилегированные гоблины встают ровно в её середину, причем при нечетной длине очереди они встают сразу за центром (*i). Шаманы принимают первого гоблина из начала очереди (-). Требуется написать программу, которая отслеживает порядок гоблинов. Входные данные содержат N запросов (до 10^5).

Ограничение времени: 0.6 секунд.

Ограничение памяти: 256 Мб.

Решение

Создаем два дека: s_1 - первая часть очереди, s_2 - вторая. Поддерживаем инвариант: $|s_1| = |s_2| + 1$. Тогда $head(s_1)$ - первый гoblin в очереди, $tail(s_2)$ - последний, $head(s_2)$ - середина при четной длине и первый элемент после центра при нечетной.

Доказательство

Очевидно, что начало s_1 соответствует первому гoblinу в очереди, а конец s_2 - последнему. Если длины деков равны и обрабатываемая операция вида *, то после выполнения блока *elif* и балансировки вставленный элемент будет в конце s_1 , что действительно соответствует середине очереди. Если же $|s_1| - 1 = |s_2|$, то вставляемый элемент окажется в начале s_2 , что соответствует первому элементу после центрального.

Алгоритмическая сложность

- По времени: $O(n)$
- По памяти: $O(n)$

Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(nullptr);
8
9     int n;
10    cin >> n;
11
12    deque<int> s1;
13    deque<int> s2;
14
15    for (int i = 0; i < n; ++i) {
16        char type;
```

```

17     cin >> type;
18
19     if (type == '+') {
20         int id;
21         cin >> id;
22         s2.push_back(id);
23     } else if (type == '*') {
24         int id;
25         cin >> id;
26         s2.push_front(id);
27     } else {
28         cout << s1.front() << "\n";
29         s1.pop_front();
30     }
31
32     if (s2.size() > s1.size()) {
33         int x = s2.front();
34         s2.pop_front();
35         s1.push_back(x);
36     }
37 }
38
39 return 0;
40 }

```

Задача №11 «К. Менеджер памяти-1»

Описание

Менеджер управляет памятью из N последовательных ячеек ($1 \leq N \leq 2^{31} - 1$). Поступает M запросов ($1 \leq M \leq 10^5$). Запрос на выделение ($K > 0$) требует найти самый левый свободный блок из K ячеек подряд, занять его и вернуть номер первой ячейки. Если такого нет - вернуть -1. Запрос на освобождение ($-T$, где $T > 0$) требует освободить память, выделенную при обработке запроса с номером T . Гарантируется, что запрос T был на выделение; если он был отклонён, текущий запрос игнорируется.

Ограничение времени: 1 секунда.

Ограничение памяти: 64 Мб.

Решение

Используем динамическое ДО. Изначально создается только корневой узел, отвечающий за весь отрезок $[1, N + 1)$. Вершины-потомки выделяются в памяти только при непосредственном спуске к ним. Каждый узел дерева хранит следующую информацию о своем отрезке:

$pref$ — длина максимального блока свободных ячеек, примыкающего к левому краю.

$suff$ — длина максимального блока свободных ячеек, примыкающего к правому краю.

mid — максимальная длина свободного блока где-либо внутри отрезка.

s — метка: 1 (весь отрезок занят), 0 (весь отрезок свободен) или -1 (смешанное состояние).

Для запроса выделения памяти ищется самый левый блок нужного размера. Если mid корня меньше K , возвращается -1. Иначе алгоритм рекурсивно спускается вниз, отдавая приоритет левому поддереву, затем проверке стыка поддеревьев (суффикс левого + префикс правого), и потом уходит в правое поддерево. Найдя подходящий индекс, мы применяем групповую операцию присваивания (1) на отрезок $[idx, idx + K)$ и сохраняем информацию о запросе в массив истории. При запросе на освобождение мы достаем из истории индекс начала и длину блока, после чего делаем групповое присваивание (0) на этот же отрезок.

Доказательство

Максимальный свободный блок внутри родительского отрезка всегда является максимумом из трех величин: самого большого блока целиком в левом ребенке, самого большого блока целиком в правом ребенке, либо блока, образованного на их границе. При обращении к несуществующим детям алгоритм корректно считает их отрезки полностью свободными. Алгоритм находит самый левый подходящий блок строгому порядку проверок при спуске.

Массовое обновление с помощью проталкивания гарантирует, что перекраска отрезков выполняется за $O(\log N)$, не требуя обхода всех листьев.

Алгоритмическая сложность

- **По времени:** $O(M \log N)$. Глубина ДО $O(\log N)$. Запрос *find* опускается по дереву, что занимает $O(\log N)$. Операции выделения и освобождения разбивают отрезок запроса на не более чем $O(\log N)$ базовых отрезков дерева. Итоговое время обработки M запросов составляет $O(M \log N)$.
- **По памяти:** $O(M \log N)$. Изначально дерево содержит 1 узел. При каждом запросе изменения/поиска алгоритм спускается на глубину ≈ 31 , создавая в худшем случае $O(\log N)$ новых узлов. Таким образом, суммарно будет создано не более $M \log N$ узлов. В реализации используется заранее выделенный пул узлов фиксированного размера ($2 \cdot 10^6$), что требует порядка 48 Мб памяти. Массив истории запросов требует дополнительно $O(M)$ памяти.

Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 struct Node {
6     int pref, suff, mid, s, l, r;
7 };
8
9 const int MAX = 2000000;
10 Node tree[MAX];
11 int cnt = 2;
12
13 int new_node(int len) {
14     int id = cnt++;
15     tree[id] = {len, len, len, -1, 0, 0};
16     return id;
17 }
18
19 void push(int x, long long lx, long long rx) {
20     if (tree[x].s == -1)
21         return;
22
23     long long mx = lx + (rx - lx) / 2;
24
25     if (!tree[x].l)
26         tree[x].l = new_node(mx - lx);
27     if (!tree[x].r)
28         tree[x].r = new_node(rx - mx);
29
30     int l = tree[x].l;
31     int r = tree[x].r;
32
33     tree[l].s = tree[x].s;
34     tree[r].s = tree[x].s;
35
36     if (tree[x].s == 1) {
37         tree[l].pref = tree[l].suff = tree[l].mid = 0;
38         tree[r].pref = tree[r].suff = tree[r].mid = 0;
39     } else {
40         tree[l].pref = tree[l].suff = tree[l].mid = mx - lx;
41         tree[r].pref = tree[r].suff = tree[r].mid = rx - mx;
```



```

42 }
43
44 tree[x].s = -1;
45 }
46
47 void pull(int x, long long lx, long long rx) {
48     long long mx = lx + (rx - lx) / 2;
49
50     int l = tree[x].l ? tree[x].l : 0;
51     int r = tree[x].r ? tree[x].r : 0;
52
53     int l_pref = l ? tree[l].pref : (mx - lx);
54     int l_suff = l ? tree[l].suff : (mx - lx);
55     int l_mid = l ? tree[l].mid : (mx - lx);
56
57     int r_pref = r ? tree[r].pref : (rx - mx);
58     int r_suff = r ? tree[r].suff : (rx - mx);
59     int r_mid = r ? tree[r].mid : (rx - mx);
60
61     tree[x].pref = l_pref;
62     if (l_pref == mx - lx)
63         tree[x].pref += r_pref;
64
65     tree[x].suff = r_suff;
66     if (r_suff == rx - mx)
67         tree[x].suff += l_suff;
68
69     tree[x].mid = max({l_mid, r_mid, l_suff + r_pref});
70 }
71
72 void op(int x, long long lx, long long rx, long long l, long long r, int p)
73 {
74     if (lx >= r || rx <= l)
75         return;
76
77     if (lx >= l && rx <= r) {
78         tree[x].s = p;
79         if (p == 1)
80             tree[x].pref = tree[x].suff = tree[x].mid = 0;
81         else
82             tree[x].pref = tree[x].suff = tree[x].mid = rx - lx;
83         return;
84     }
85
86     push(x, lx, rx);
87
88     long long mx = lx + (rx - lx) / 2;
89
90     if (!tree[x].l)
91         tree[x].l = new_node(mx - lx);
92     if (!tree[x].r)
93         tree[x].r = new_node(rx - mx);
94
95     op(tree[x].l, lx, mx, l, r, p);
96     op(tree[x].r, mx, rx, l, r, p);
97
98     pull(x, lx, rx);
99 }
100 int find(int x, long long lx, long long rx, int k) {

```

```

101 if (tree[x].mid < k)
102     return -1;
103
104 if (rx - lx == k)
105     return lx;
106
107 push(x, lx, rx);
108
109 long long mx = lx + (rx - lx) / 2;
110
111 int l = tree[x].l ? tree[x].l : 0;
112 int r = tree[x].r ? tree[x].r : 0;
113
114 int l_mid = l ? tree[l].mid : (mx - lx);
115 int l_suff = l ? tree[l].suff : (mx - lx);
116 int r_pref = r ? tree[r].pref : (rx - mx);
117
118 if (l_mid >= k) {
119     if (!tree[x].l)
120         tree[x].l = new_node(mx - lx);
121     return find(tree[x].l, lx, mx, k);
122 }
123
124 if (l_suff + r_pref >= k)
125     return mx - l_suff;
126
127 if (!tree[x].r)
128     tree[x].r = new_node(rx - mx);
129
130 return find(tree[x].r, mx, rx, k);
131 }
132
133 struct Req {
134     int start, len;
135 };
136
137 int main() {
138     ios_base::sync_with_stdio(false);
139     cin.tie(nullptr);
140
141     int n, m;
142     cin >> n >> m;
143
144     tree[1].pref = tree[1].suff = tree[1].mid = n;
145     tree[1].s = -1;
146
147     vector<Req> history(m + 1);
148
149     int y;
150     for (int i = 1; i <= m; i++) {
151         cin >> y;
152
153         if (y > 0) {
154             int z = find(1, 1, (long long)n + 1, y);
155
156             if (z == -1) {
157                 cout << "-1\n";
158                 history[i] = {-1, 0};
159             } else {
160                 cout << z << "\n";

```

```

161     history[i] = {z, y};
162     op(1, 1, (long long)n + 1, z, (long long)z + y, 1);
163 }
164 } else {
165     int idx = -y;
166
167     if (idx <= m && history[idx].start != -1) {
168         op(1,
169             1,
170             (long long)n + 1,
171             history[idx].start,
172             (long long)history[idx].start + history[idx].len,
173             0);
174         history[idx] = {-1, 0};
175     }
176
177     history[i] = {-1, 0};
178 }
179 }
180
181 return 0;
182 }

```

Задача №12 «L. Минимум на отрезке»

Описание

Рассмотрим последовательность целых чисел длины N ($1 \leq N \leq 150000$). По ней с шагом 1 движется «окно» длины K ($1 \leq K \leq 10000$, $K \leq N$). Сначала в «окне» видно первые K чисел, на следующем шаге — K чисел, начиная со второго, и так далее до конца последовательности. Требуется для каждого положения «окна» определить минимум в нём. Числа последовательности не превосходят по модулю 10^5 .

Ограничение времени: 0.5 секунд.

Ограничение памяти: 256 Мб.

Решение

Строим дерево отрезков на минимум. Если размер массива не степень двойки, то добавляем до ближайшей степени массив нейтральным элементом по взятию минимума. Далее строим дерево, каждый узел хранит минимум в соответствующем отрезке. Таким образом, если отрезок запроса не пересекается с отрезком текущего узла, то возвращаем нейтральный элемент. Если отрезок узла лежит внутри отрезка запроса, то возвращаем значение узла, если пересекается - запускаемся от обоих детей.

Доказательство

Рассмотрим дерево отрезков. Отрезок запроса можно покрыть отрезками узлов дерева. Тогда можно рассмотреть минимальное покрытие (покрытий конечное количество, минимальное в смысле количества отрезков дерева, которые покрывают отрезок запроса). Заметим, что когда рекурсия дойдет до соответствующих минимальному покрытию узлов дерева - будет возвращен минимум на этом отрезке. Значит во время обхода будет возвращен либо минимум на отрезке из покрытия либо нейтральный элемент, а так как операция взятия минимума коммутативна и ассоциативна, то от порядка ничего не зависит.

Алгоритмическая сложность

- По времени: $O(n \log(n))$
- По памяти: $O(n)$

Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 void build(int* t, int x, int lx, int rx, int* arr) {
6     if (rx - lx == 1) {
7         t[x] = arr[lx];
8         return;
9     }
```

```

10  int mx;
11  mx = (rx + lx) / 2;
12  build(t, 2 * x + 1, lx, mx, arr);
13  build(t, 2 * x + 2, mx, rx, arr);
14  t[x] = min(t[2 * x + 1], t[2 * x + 2]);
15 }
16
17 int getmin(int* t, int x, int lx, int rx, int l, int r) {
18     if (lx >= l && rx <= r) {
19         return t[x];
20     }
21     if (lx >= r || rx <= l) {
22         return INT32_MAX;
23     }
24     int mx;
25     mx = (lx + rx) / 2;
26     int a;
27     a = getmin(t, 2 * x + 1, lx, mx, l, r);
28     int b;
29     b = getmin(t, 2 * x + 2, mx, rx, l, r);
30     return min(a, b);
31 }
32
33 int main() {
34     ios_base::sync_with_stdio(false);
35     cin.tie(nullptr);
36
37     int n;
38     int k;
39     cin >> n >> k;
40     vector<int> arr(n);
41     for (int i = 0; i < n; i++) {
42         cin >> arr[i];
43     }
44     int l;
45     l = 1;
46     while (l < n) {
47         l *= 2;
48     }
49     vector<int> m(l);
50     for (int i = 0; i < n; i++) {
51         m[i] = arr[i];
52     }
53     for (int i = n; i < l; i++) {
54         m[i] = INT32_MAX;
55     }
56     vector<int> tree(2 * l);
57     build(tree.data(), 0, 0, l, m.data());
58
59     for (int i = k - 1; i < n; i++) {
60         cout << getmin(tree.data(), 0, 0, l, i - k + 1, i + 1) << " ";
61     }
62     return 0;
63 }

```